

# Module 6.2: Hive

## Contents

|                                        |   |
|----------------------------------------|---|
| Apache Hive .....                      | 1 |
| Hive Data Types.....                   | 2 |
| Hive File Formats.....                 | 3 |
| Similarities between Pig and Hive..... | 6 |
| Differences between Pig and Hive ..... | 6 |

## Apache Hive

Apache Hive is a data warehouse and SQL-like query tool built on top of Hadoop. It allows users to analyze and manage large datasets stored in HDFS using HiveQL, a language similar to SQL. Hive converts queries into MapReduce, Tez, or Spark jobs for execution.

- Provides SQL-like query language (HiveQL).
- Supports large-scale data processing.
- Runs on top of Hadoop and HDFS.
- Supports partitioning and bucketing.
- Allows schema-on-read data management.
- Supports various file formats such as ORC, Parquet, and Avro.
- Extensible through User Defined Functions (UDFs).

### Advantages of Hive

- Easy SQL-like querying of big data.
- Eliminates complex MapReduce programming.
- Scalable for petabyte-scale datasets.
- Supports data warehousing operations.
- Integrates with Hadoop ecosystem tools.
- Supports multiple file formats.

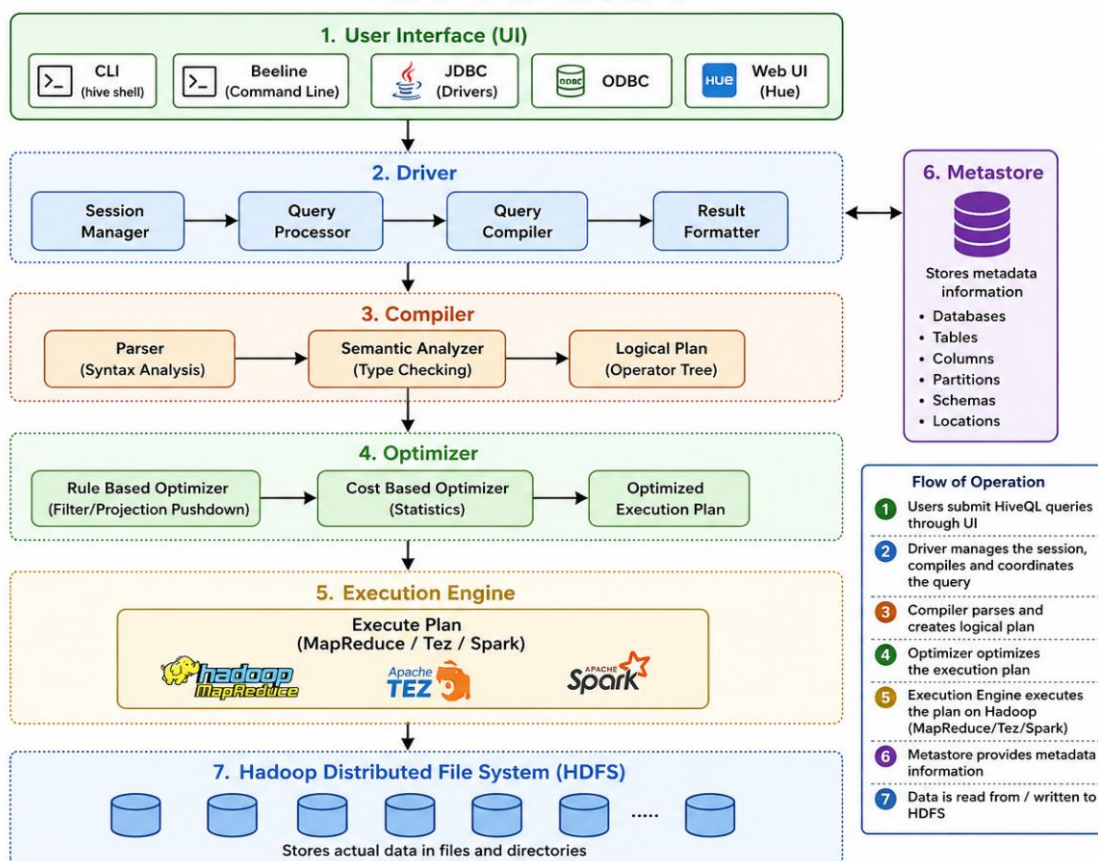
### Applications of Hive

- Data warehousing.
- Business intelligence reporting.
- Log file analysis.
- ETL (Extract, Transform, Load) processing.
- Data summarization.
- Big data analytics.

### Hive Architecture

1. **User Interface (UI):** Provides access to Hive through tools like CLI, Beeline, JDBC, or Web UI.
2. **Driver:** Receives HiveQL queries, manages user sessions, and controls the overall execution.
3. **Compiler:** Parses queries, checks syntax rules, and generates a logical execution plan.
4. **Optimizer:** Modifies and refines the execution plan to deliver better overall performance.
5. **Execution Engine:** Runs the optimized plan using frameworks like MapReduce, Tez, or Spark.
6. **Metastore:** Stores metadata including table names, column schemas, partitions, and data locations.
7. **Hadoop Ecosystem:** Connects directly with HDFS for data storage and Hadoop for distributed processing.

## Hive Architecture



## Hive Data Types

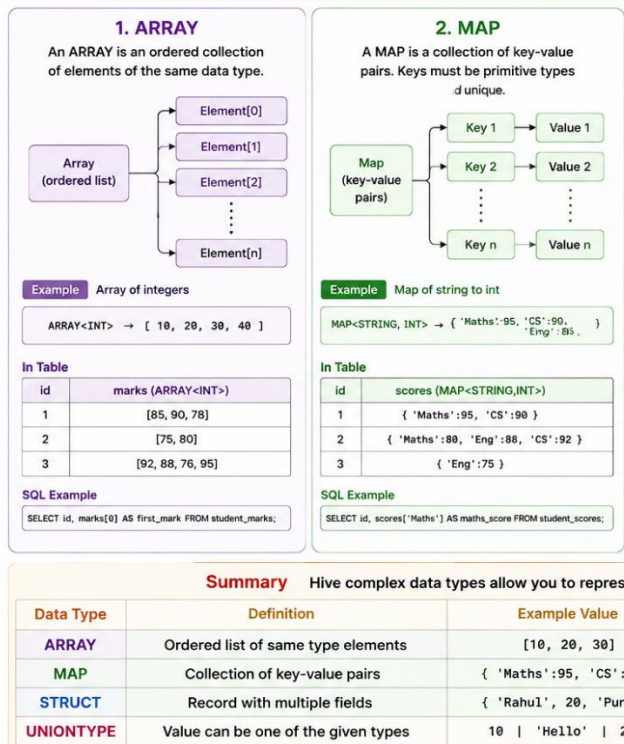
Hive supports both primitive and complex data types.

### 1. Primitive Data Types

| Data Type | Description              | Example             |
|-----------|--------------------------|---------------------|
| TINYINT   | 1-byte integer           | 10                  |
| SMALLINT  | 2-byte integer           | 200                 |
| INT       | 4-byte integer           | 1000                |
| BIGINT    | 8-byte integer           | 100000              |
| FLOAT     | Single precision decimal | 12.5                |
| DOUBLE    | Double precision decimal | 25.678              |
| DECIMAL   | Exact numeric value      | 123.45              |
| BOOLEAN   | True or False            | TRUE                |
| STRING    | Character string         | "Hive"              |
| CHAR      | Fixed length string      | CHAR(10)            |
| VARCHAR   | Variable length string   | VARCHAR(50)         |
| DATE      | Date value               | 2025-01-01          |
| TIMESTAMP | Date and time            | 2025-01-01 10:00:00 |

## Hive Complex

Hive supports complex data types that allow you to store colle



## Hive File Formats

Hive supports several storage formats for efficient data processing.

- Text File:** Default storage format in Hive where data is stored as plain text.
  - Human readable.
  - Easy to create and edit.
  - Larger storage size.
  - Slower query performance.
- Sequence File:** Binary key-value file format used in Hadoop.
  - Stores data in compressed form.
  - Faster than text files.
  - Suitable for MapReduce jobs.
- ORC File (Optimized Row Columnar):** A highly optimized columnar storage format developed for Hive.
  - High compression.
  - Faster query execution.
  - Supports indexing.
  - Reduces storage requirements.

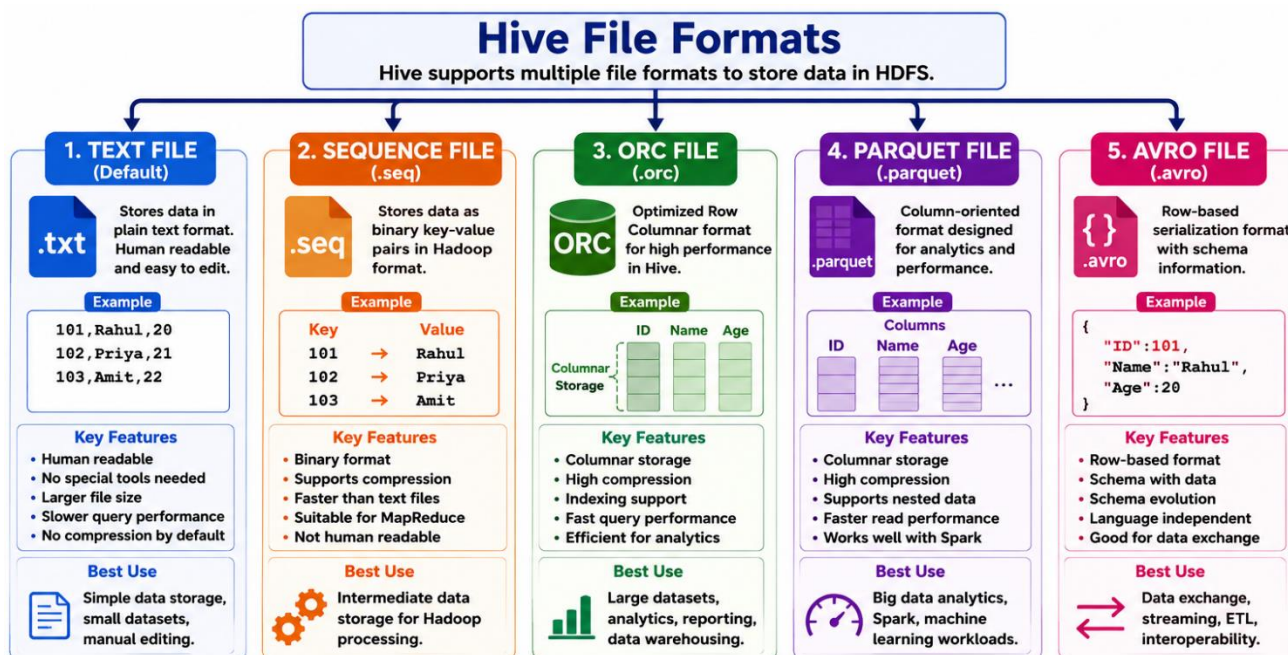
**Advantages**
- Avro File:** A row-based data serialization format developed by Apache.
  - Schema stored with data.
  - Supports schema evolution.
  - Language independent.

**Advantages**

  - Good for data exchange between systems.
  - Suitable for streaming and ETL applications.
- Parquet File:** A column-oriented storage format widely used in big data systems.
  - Efficient compression.
  - Faster analytical processing.
  - Supports complex nested data structures.

**Advantages**

  - Works efficiently with Hive and Spark.
  - Reduces disk I/O.



| Format        | Storage Type | Compression | Query Speed | Best Use               |
|---------------|--------------|-------------|-------------|------------------------|
| Text File     | Row          | Low         | Slow        | Simple data storage    |
| Sequence File | Key-Value    | Medium      | Moderate    | Hadoop processing      |
| ORC           | Columnar     | High        | Very Fast   | Hive analytics         |
| Parquet       | Columnar     | High        | Very Fast   | Hive & Spark analytics |
| Avro          | Row          | Medium      | Moderate    | Data exchange & ETL    |

# HiveQL DDL, DML and Queries

## 6.3.4 HIVEQL DATA DEFINITION LANGUAGE (DDL)

### 6.3.4.1 Creating Databases

**Syntax**

```
CREATE DATABASE database_name;
```

**Example**

```
CREATE DATABASE CollegeDB;
```

**Verify**

```
SHOW DATABASES;
```

### 6.3.4.2 Creating Tables

**Syntax**

```
CREATE TABLE table_name (  
  column1 datatype,  
  column2 datatype,  
  column3 datatype  
);
```

**Example**

```
CREATE TABLE Student (  
  ID INT,  
  Name STRING,  
  Age INT  
);
```

View Structure: DESCRIBE Student;

### 6.3.4.3 Altering Tables

**Add Column**

```
ALTER TABLE Student  
ADD COLUMNS (Department STRING);
```

**Rename Table**

```
ALTER TABLE Student  
RENAME TO Student_Info;
```

**Change Column Name**

```
ALTER TABLE Student_Info  
CHANGE Age Student_Age INT;
```

### 6.3.4.4 Dropping Databases and Tables

**Drop Table**

```
DROP TABLE Student_Info;
```

**Drop Database**

```
DROP DATABASE CollegeDB;
```

**Force Drop Database**

```
DROP DATABASE CollegeDB CASCADE;
```

**Purpose:** Used to permanently remove tables and databases from Hive.

### 6.3.5.1 Loading Data

**Syntax**

```
LOAD DATA INPATH 'filepath'  
INTO TABLE table_name;
```

**Example**

```
LOAD DATA INPATH '/user/hadoop/student.txt'  
INTO TABLE Student;
```

**Purpose:** Imports external data into Hive tables.

### 6.3.5.2 Inserting Data

**Insert Single Record**

```
INSERT INTO Student  
VALUES (101, 'Rahul', 20);
```

**Insert Multiple Records**

```
INSERT INTO Student VALUES  
(102, 'Priya', 21),  
(103, 'Amit', 22);
```

**Purpose:** Adds new records into a table.

### 6.3.5.3 Updating Data

(Update supported in ACID transactional tables)

**Example**

```
UPDATE Student  
SET Age = 21  
WHERE ID = 101;
```

**Purpose:** Modifies existing records.

### 6.3.5.4 Deleting Data

(Delete supported in ACID transactional tables)

**Example**

```
DELETE FROM Student  
WHERE ID = 101;
```

**Purpose:** Removes unwanted records from a table.

## 6.3.6 HIVEQL QUERIES

### 6.3.6.1 SELECT Query

**Example**

```
SELECT * FROM Student;
```

**Output**

| ID  | Name  | Age |
|-----|-------|-----|
| 101 | Rahul | 20  |
| 102 | Priya | 21  |
| 103 | Amit  | 22  |

**Purpose:** Retrieves data from a table.

### 6.3.6.2 WHERE Clause

**Example**

```
SELECT * FROM Student  
WHERE Age > 20;
```

**Output**

| ID  | Name  | Age |
|-----|-------|-----|
| 102 | Priya | 21  |
| 103 | Amit  | 22  |

**Purpose:** Filters records based on condition.

### 6.3.6.3 GROUP BY Clause

**Example**

```
SELECT Department, COUNT(*)  
FROM Employee  
GROUP BY Department;
```

**Output**

| Department | Count |
|------------|-------|
| CSE        | 50    |
| ECE        | 40    |

**Purpose:** Groups rows having the same values.

### 6.3.6.4 ORDER BY Clause

**Example**

```
SELECT * FROM Student  
ORDER BY Age DESC;
```

**Output**

| ID  | Name  | Age |
|-----|-------|-----|
| 103 | Amit  | 22  |
| 102 | Priya | 21  |
| 101 | Rahul | 20  |

**Purpose:** Sorts query results.

### 6.3.6.5 JOIN Operations

**Example (INNER JOIN)**

```
SELECT S.ID, S.Name, M.Marks  
FROM Student S  
JOIN Marks M  
ON S.ID = M.ID;
```

**Output**

| ID  | Name  | Marks |
|-----|-------|-------|
| 101 | Rahul | 85    |
| 102 | Priya | 90    |

**Purpose:** Combines rows from two or more tables.

### 6.3.6.6 Aggregate Functions

**Example**

```
SELECT  
COUNT(*) AS Total_Students,  
AVG(Age) AS Avg_Age,  
MAX(Age) AS Max_Age,  
MIN(Age) AS Min_Age  
FROM Student;
```

**Output**

| Total_Students | Avg_Age | Max_Age | Min_Age |
|----------------|---------|---------|---------|
| 3              | 21.00   | 22      | 20      |

**Purpose:** Performs calculations on multiple rows.

## Summary

**DDL:** Defines structure (DB, Tables)

**DML:** Manages data (Load, Insert, Update, Delete)

**Queries:** Retrieves and analyzes data

HiveQL is SQL-like language used to interact with Hive data stored in HDFS.



## Similarities between Pig and Hive

Apache Pig and Apache Hive are high-level data processing tools developed for the Hadoop ecosystem. Both simplify big data processing by providing abstraction over MapReduce programming.

1. **Built on Hadoop:** Both Pig and Hive run on Hadoop and use HDFS for data storage. They convert user commands into MapReduce, Tez, or Spark jobs.
2. **Handle Large Data Sets:** Designed to process and analyze massive volumes of structured, semi-structured, and unstructured data.
3. **Reduce Coding Complexity:** Eliminate the need to write complex Java MapReduce programs.
4. **Support Distributed Processing:** Data is processed across multiple nodes in a Hadoop cluster.
5. **Extensible:** Both support User Defined Functions (UDFs) for custom operations.
6. **Fault Tolerant:** Inherit Hadoop's fault tolerance and reliability features.
7. **Open Source:** Both are Apache open-source projects and freely available.
8. **Data Transformation Capabilities:** Provide operations for filtering, grouping, sorting, and joining data.

## Differences between Pig and Hive

| Feature           | Pig                               | Hive                              |
|-------------------|-----------------------------------|-----------------------------------|
| Developed By      | Yahoo                             | Facebook                          |
| Language Used     | Pig Latin                         | HiveQL (SQL-like language)        |
| User Type         | Programmers and Developers        | Data Analysts and SQL Users       |
| Programming Style | Procedural                        | Declarative                       |
| Learning Curve    | Requires scripting knowledge      | Easy for SQL users                |
| Data Processing   | Describes how data flows          | Describes what result is needed   |
| Query Language    | Pig Latin                         | HiveQL                            |
| Schema            | Schema can be applied when needed | Schema is defined before querying |
| Best For          | ETL and data transformation       | Data warehousing and reporting    |
| Optimization      | Limited automatic optimization    | Built-in query optimization       |
| Database Support  | No database concepts              | Supports databases and tables     |
| Output            | Processed datasets                | Query results and reports         |
| Ease of Use       | More flexible for programmers     | More user-friendly for analysts   |

### Use Cases of Pig

1. **ETL (Extract, Transform, Load) Operations:** Cleaning and transforming raw data before loading into databases.
2. **Log File Processing:** Analyzing web server logs and application logs.
3. **Data Integration:** Combining data from multiple sources.
4. **Data Preparation:** Preparing datasets for machine learning and analytics.
5. **Large-Scale Data Transformation:** Filtering, grouping, sorting, and aggregating large datasets.

### Example Applications

- Social media data analysis
- Clickstream analysis
- Data cleansing pipelines
- Recommendation system preprocessing

### Use Cases of Hive

1. **Data Warehousing:** Storing and managing structured big data.
2. **Business Intelligence (BI):** Generating reports and dashboards.
3. **Data Analysis:** Querying large datasets using SQL-like commands.
4. **Reporting Systems:** Creating periodic business reports.
5. **Decision Support Systems:** Analyzing historical data for strategic decisions.

### Example Applications

- Sales reporting
- Customer analytics
- Financial analysis
- Data warehouse management
- Business dashboard

